

Red first. Then green. Then a pull request.

How to use the tdd pipeline to take a one-line request to a reviewed spec, a failing test suite, an implementation, an adjudicated review and a shipped branch, on Claude Code or Cursor.

EXAMPLE RUN · ABRIDGED

terminal · my-app

> /tdd:task "validate input types on add"

TASK-1 spec written · review recorded

gate you answered: proceed

worktree .tdd-pipeline/worktrees/TASK-1

test Status: fail (red) 4 tests

execute Tests: pass (4) Lint: clean

code-review Verdict: APPROVED

end End phase: completed

Result: SHIPPED PR opened against main

A plugin that runs your coding agent as a test-first pipeline

tdd adds eight slash commands to Claude Code or Cursor. You describe the work in one line; the pipeline plans it, writes tests before code, implements, reviews and opens a pull request.

	PLAN	REVIEW	GATE	BUILD	SHIP
You	<code>/tdd:plan "validate input types on add"</code> , then answer its questions and confirm the spec	<code>/tdd:review TASK-1</code> , then <code>/tdd:task TASK-1</code> to start the work	Read the spec. Say “proceed”, or stop and keep it as a plan	Nothing to do	Review and merge the pull request
Pipeline	Resolves <code>TASK-1</code> , explores the code, drafts the spec, argues against it, commits it	A fresh reader critiques the spec against the real code and fixes it in place	Waits for your answer	Failing tests, implementation, code review: each a fresh agent, in a worktree	Lints, pushes, opens the PR, updates the tracker

Shortcut: `/tdd:task "validate input types on add"` runs plan and review in one go, with a single grouped question, then stops at the same gate. Either way, the project needs a test command that runs and a conventions file; `init` sets up both.

Coding agents write plausible code. Plausible is not the same as correct.

Tests written after the code confirm the code

An agent that implements first writes tests describing what it built, not what was asked. The suite goes green and proves nothing.

An agent grading its own work passes it

The context that made a mistake is the worst place to find it. Self-review in the same session repeats the same assumptions.

Plans rest on guesses about your code

“Extend the existing endpoint” or “the API returns X”, never checked, turns into a whole branch built on a wrong assumption.

State kept in a chat dies with the chat

A session that crashes or runs out of context takes its decisions with it, and the next one starts over.

tdd makes these failures structurally hard instead of relying on a careful prompt: tests first, a fresh agent per phase, specs reviewed against the code, and all state in files.

What you get from it

Tests that failed before the code existed

The test phase has to see the suite go red, for the reason each test asserts, before execute writes a line.

TEST PHASE · FAIL (RED)

Tests nobody quietly weakened

Execute may not edit an assertion or delete a test, and code-review diffs every test file against the commit that wrote it.

TEST PROTECTION CONTRACT

Plans checked against the real code

Before the gate, a fresh reviewer verifies the spec's claims about your code and external APIs. A guess is a blocker, not a note.

REVIEW · HARD BLOCKERS

Your rules, enforced with citations

Execute cites the conventions it applied by `file:line`, and code-review checks each citation against the file.

CONVENTIONS · CODE-REVIEW

Work that survives the session

The spec, its handoff log and a branch per task hold all state, so any phase can resume later, even from another editor.

HANDOFF LOG · RESUME

Your attention where it counts

You decide twice: at the gate, after reading the spec, and on the pull request. Everything in between runs on its own.

GATE · PULL REQUEST

The package holds the mechanics; your project holds the facts

PHASE	IN THE PACKAGE	IN YOUR PROJECT
pipeline	Worktree guard, sequencing of the four phases, result aggregation	Base branch, worktree directory, <code>git.worktree_setup</code> script
test	Study existing test patterns, red-phase verification, handoff log, commit	<code>commands.test</code> , <code>paths.tests</code> , test and mocking patterns from your conventions
execute	Test protection contract, self-review, verification loop, lint gate, 300-character report	Architecture rules from your conventions, <code>commands.test_all</code> , <code>commands.lint</code>
code-review	Spec compliance, test integrity and quality, security, three-level verdict	Layer-specific rules from your conventions, optional <code>paths.review_checklist</code>
end	Lint, persist discoveries, push, PR, tracker status	<code>commands.lint</code> , <code>pr.enabled</code> , <code>tracker.type</code>

The trade-off. An inline rule is always read; a pointer to a conventions file depends on the agent finding the right section. So `execute` cites the exact convention lines it applied, `code-review` checks every citation, and an area with no rule becomes an explicit warning.

One harness-neutral core, with thin adapters that point at it

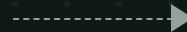
Claude Code

`/tdd:task``adapters/claude-code/commands/task.md`

Follow `${CLAUDE_PLUGIN_ROOT}/core/flows/task.md`

Namespaced by the plugin, so it never clashes with the built-in `/init` or `/review`.

POINTS AT



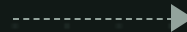
Cursor

`/tdd-task``adapters/cursor/commands/tdd-task.md`

Follow `$PIPELINE_ROOT/core/flows/task.md`

Root from `TDD_PIPELINE_ROOT`, the project's `.cursor/tdd`, or the user-level plugin.

POINTS AT



core/

PLAIN MARKDOWN · THE ONLY BEHAVIOUR

└ contracts/

7 Config schema, spec template, handoff log, conventions template, review checklist

└ flows/

8 init, conventions, doctor, **task**, plan, review, fix-bug, resume

└ phases/

5 pipeline (orchestrator), test, execute, code-review, end

└ trackers/

3 none, github, linear

enforced

`checks/core-is-neutral.sh` fails if core names a harness or a harness tool, so a project keeps working when a developer switches editors.

Eight commands, typed differently on each harness

COMMAND	CLAUDE CODE	CURSOR	WHAT IT DOES
SET UP A PROJECT			
init	<code>/tdd:init</code>	<code>/tdd-init</code>	Detects the stack, proposes <code>.tdd-pipeline/config.yaml</code> , commits it, hands off to conventions. Once per project.
conventions	<code>/tdd:conventions</code>	<code>/tdd-conventions</code>	Derives the project's rules from the code, each with a <code>file:line</code> citation.
doctor	<code>/tdd:doctor</code>	<code>/tdd-doctor</code>	Checks every requirement: <code>pass</code> <code>fail</code> <code>n/a</code> . Add <code>repair</code> to fix what it can.
DO WORK			
task	<code>/tdd:task</code>	<code>/tdd-task</code>	The main entry point: spec → review → gate → worktree → four phases → PR.
plan	<code>/tdd:plan</code>	<code>/tdd-plan</code>	Designs a change under a task ID and stops; <code>task</code> picks it up later.
fix-bug	<code>/tdd:fix-bug</code>	<code>/tdd-fix-bug</code>	Reproduces a defect with a failing test, finds the root cause, makes the minimal fix. Stops before pushing.
REVIEW AND RECOVER			
review	<code>/tdd:review</code>	<code>/tdd-review</code>	Critiques a spec against the real codebase and fixes it in place.
resume	<code>/tdd:resume</code>	<code>/tdd-resume</code>	Re-enters an interrupted pipeline at one phase, runs only that phase, stops.

Install once per machine, then prepare each project with init

Claude Code · plugin marketplace

terminal

```
claude plugin marketplace add dfrnks/agent-plugins
claude plugin install tdd@dfrnks

# verify: should report Skills (8), Agents (5)
claude plugin details tdd@dfrnks
```

`marketplace add` takes the GitHub path. `install` takes the marketplace's own name, `dfrnks`. The mismatch is intentional.

Adding the marketplace from a local clone path means edits to the clone take effect on the next dispatch, which is useful when you work on the pipeline itself.

Cursor · user-level plugin

terminal

```
git clone https://github.com/dfrnks/agent-plugins.git \
~/src/agent-plugins
ln -s ~/src/agent-plugins ~/.cursor/plugins/local/tdd
# then: Developer: Reload Window
```

Cursor · per project

```
./install.sh --harness cursor --project /path/to/project
```

Both can coexist. An adapter prefers a project's own copy of `core/`, so a project can pin its own revision. Updating is a `git pull` in the clone.

Permissions: when one is missing, it doesn't look like a permission problem

The plugin root must be readable

Every command reads files under the package's `core/`, outside your project. Interactive sessions ask once. Headless runs need it passed in:

```
claude -p "" --add-dir /path/to/agent-plugins
```

The configured commands must run

If the test phase cannot run `commands.test`, it reports `unverified` instead of claiming red, and the run stops as `BLOCKED`.

git add and git commit must be allowed

`init` and `conventions` commit what they write, because a worktree only contains committed files. If git is denied, the files get written, the commits don't, and doctor fails on the "tracked in git" rows.

The trap on Claude Code: trust the workspace first. In an untrusted workspace, Claude Code ignores the *whole* `permissions.allow` list in `.claude/settings.json`. Commits fail silently and doctor reports "not ready". Open the project interactively once and accept the trust dialog.

init proposes, you confirm

- 1 Detect the stack: test runner, linter, base branch, git-ignored dependencies.
- 2 Propose the config and ask about everything it could not detect: tracker, conventions path, pull requests, how often the full suite runs.
- 3 Create directories under `.tdd-pipeline/` and git-ignore the worktrees directory.
- 4 Write a worktree setup script if dependencies are git-ignored (a venv, `node_modules`, a local `.env`).
- 5 Derive conventions by handing off to the conventions flow.
- 6 Verify the result with doctor.

`.tdd-pipeline/config.yaml` · Python example

```
version: 1
project: ledger
tracker:
  type: none           # none | linear | github
  prefix: TASK         # IDs and branch names: TASK-1, TASK-2
commands:
  test: ".venv/bin/python -m pytest -q {target}"
  test_all: ".venv/bin/python -m pytest -q"
  lint: ".venv/bin/python -m ruff check ."
paths:
  tests: [tests]
  specs: .tdd-pipeline/tasks
  worktrees: .tdd-pipeline/worktrees
  conventions: CONVENTIONS.md # no default
git:
  base_branch: main
  commit_trailer: ""
  worktree_setup: scripts/setup-worktree.sh
pr:
  enabled: true
policy:
  full_suite: on_execute_and_end # where test_all runs
```

The pipeline stops on a missing key. It never guesses.

Every phase reads the config at Step 0. It never infers a command or a path from a package manifest or a directory listing: a wrong guess in an unattended run costs a whole branch of invalid work.

```
phase output · Step 0
```

```
Cannot start: `.tdd-pipeline/config.yaml` is missing  
key `commands.test`. Add it, or run the init flow  
to regenerate the configuration.
```

Mode-specific keys follow the same rule. `tracker.type: linear` also needs `tracker.team` and `tracker.states`.

`paths.conventions` has no default

Point it at the file that already holds your rules. Execute and code-review enforce whatever it contains, so an empty file means they enforce nothing.

`git.worktree_setup`

Rebuilds what git doesn't track inside each fresh worktree. It runs by its own path, so it needs a shebang and the executable bit.

`git.commit_trailer: ""`

The only trailer any flow may add. Empty means no trailer at all, and no agent attribution lines either.

Choose which phases pay for the full suite

The full run, `commands.test_all`, is usually the biggest cost in a pipeline. The targeted run, `commands.test`, always gates every Definition of Done item. The policy only decides where the full run also happens.

VALUE		TEST	EXECUTE	END
every_phase	default	●	●	●
on_execute_and_end	recommended	—	●	●
on_execute		—	●	—
on_end		—	—	●
never	nowhere: CI is the regression gate			

Unset means every_phase

The slowest value, kept as the default so an upgrade never starts verifying less without anyone choosing it. doctor passes the row with a note that says so.

init measures before it asks

It times `commands.test` on one small module and `commands.test_all`, and puts both numbers in the question. A 15-minute full run is roughly an hour per task versus fifteen minutes.

Your conventions are what the pipeline enforces, so this is the step to watch

- Parallel subagents explore seven areas: layering, error handling, naming, auth, data access, tests, formatting.
- A pattern becomes a rule only when it holds at **two or more independent sites**. A single instance is reported as an observation.
- Every rule carries the `file:line` it was read from.
- You accept, edit or drop each group. There is no single yes for the whole set.
- Doctor fails the project if the managed section holds **fewer than five rules**.

FROM THIS REPOSITORY'S OWN CONVENTIONS FILE

CLAUDE.md · managed section

```
←!— pipeline:conventions:start →  
## Data access  
  
- Stage named paths; never `git add .` or  
  `git add -A` (core/flows/conventions.md:128,  
  core/phases/end.md:165).  
- Commit anything a phase must read; an  
  uncommitted file is invisible from inside a  
  worktree (core/flows/conventions.md:130).  
←!— pipeline:conventions:end →  
  
←!— pipeline:discoveries:start →  
  findings the end phase appends; never regenerated  
←!— pipeline:discoveries:end →
```

doctor reports one row per requirement and names the fix for the first failure

EXAMPLE OUTPUT · ABRIDGED

CHECK	RESULT
<code>.tdd-pipeline/config.yaml</code> , every key, tracked	pass
Git repository with base branch <code>main</code>	pass
<code>paths.specs</code> directory	pass
<code>paths.worktrees</code> directory, git-ignored	pass
<code>paths.conventions</code> , derived rules	pass
<code>policy.full_suite</code> set explicitly	pass
<code>paths.review_checklist</code> (not set)	n/a
<code>git.worktree_setup</code> (no ignored deps)	n/a

6 pass · 0 fail · 2 n/a — ready

Reading a fail

On not ready, doctor names the exact command for the first failure: `init`, `conventions` or a direct config edit. It only names a fix that can actually clear the row.

"exists but is not tracked in git"

The file was written but the commit was not. Neither `init` nor `conventions` can clear this row. Check the trust dialog and git permissions, then commit the file yourself.

Give task an ID or a sentence. Everything before the gate is read-only.

BEFORE THE GATE · ON THE BASE BRANCH

- 0 Load configuration, fail fast on a missing key.
- 1 Resolve the item through the tracker and mark it started.
- 2 Write the spec. Parallel exploration, then *one* grouped question about scope and constraints, then a draft committed as `TASK-1: add spec`.
- 3 Review the spec against the real codebase, fix it in place, and log a `### review` entry.

GATE

STEP 4
"PROCEED"?

AFTER THE GATE · IN A WORKTREE

- 5 Create the worktree on a branch named exactly `TASK-1`.
- 6 Run the pipeline: a subagent dispatches test, execute, code-review, end.
- 7 Report: task ID, spec path, review outcome, how the gate ended, result, branch, PR URL.

```
/tdd:task "validate input types on add"  
/tdd:task TASK-12      # existing item  
/tdd:task #42          # github tracker
```

Any answer other than “proceed” stops the run and leaves you a reviewed spec

The gate is a hard stop. Declining leaves a reviewed, committed spec and nothing else: no worktree, no dispatch, no tracker change. That is also the plan-only path. Running `task TASK-1` later picks up the existing spec and goes straight to review.

Split signals. If the Approach names more than five changes, or the Definition of Done has more than ten items, the task is several tasks. Split it into `TASK-1-1.md`, `TASK-1-2.md`, ... under a parent index.

The review blocks a spec that says “verify the API before implementing”. External schemas and existing data models get checked during planning, not left to the implementing phase.

SPEC SKELETON

```
.tdd-pipeline/tasks/TASK-1.md
```

```
# TASK-1 – Validate input types on add
```

```
## Context           why the change is needed
## Approach          files, functions, data sources, by name
## Files to Modify   every file, one note each
## Definition of Done
- [ ] add() raises TypeError for non-numeric input
      (convention: raise, never return None)
- [ ] Test or verification step
## Out of Scope      so no phase drifts into it
## Agent Handoff Log
<!-- phases append findings here -->
```

Each task gets its own worktree, on a branch named exactly after the task

LOCAL BASE IN SYNC WITH THE REMOTE

```
git fetch origin main
git worktree add --no-track \
    .tdd-pipeline/worktrees/TASK-1 -b TASK-1 origin/main
```

`--no-track` keeps the task branch from inheriting `main` as its upstream. Without it, a bare `git push` would suggest pushing straight onto `main`.

LOCAL BASE HAS UNPUSHED COMMITS, OR THERE'S NO REMOTE

```
git worktree add \
    .tdd-pipeline/worktrees/TASK-1 -b TASK-1 main
```

Unpushed commits often include the config itself. The report names which ref was used.

Then, inside the worktree

- The orchestrator checks it is not in the main checkout and that the branch equals the task ID.
- It checks that the spec's handoff log has a review entry.
- It runs `./scripts/setup-worktree.sh` if configured, and stops on a non-zero exit.

Parallel tasks are supported

One worktree per task, under `paths.worktrees`, which doctor requires to be git-ignored. An existing worktree with uncommitted work stops the run and asks first.

Four fresh subagents, run in order, each one gated on the last one's report

test

Writes the failing suite that defines the behaviour, and watches it fail.

MODEL opus
REPORT ≤ 300 chars
COMMITTS the tests

execute

Makes the suite pass without touching what "pass" means.

MODEL opus
REPORT ≤ 300 chars
COMMITTS the implementation

code-review

Adjudicates the diff against the spec, the conventions and the tests.

MODEL opus
REPORT ≤ 500 chars
COMMITTS its log entry

end

Lints, saves what was learned, pushes, opens the PR, updates the tracker.

MODEL sonnet
REPORT one line per item
PUSHES branch + PR

task-pipeline

(sonnet) dispatches each phase, waits for it to return, and reads one field of its report to decide whether the next runs. It never inlines a phase, never pauses mid-run, and never loops back.

The short reports are for the orchestrator. The detail goes into the spec's handoff log, which the next phase reads in its fresh context.

Red is something the phase observed, not something it assumed

- Studies existing test patterns, writes a test plan, then writes tests under `paths.tests`.
- Runs `commands.test`. Every new test must fail **for the reason it asserts**. A typo, import error or broken fixture also shows red, and proves nothing.
- A test that passes before the implementation is either a **regression guard** (kept, noted as green by design) or a test that doesn't exercise the change (rewritten).
- Under `policy.full_suite: every_phase`, also runs `commands.test_all` to confirm nothing that already passed broke.

STATUS	NEXT	MEANING
fail (red)	execute	The contract is written and currently unmet.
n/a (nothing testable)	execute	Nothing a command can observe (prose only). Code-review checks the claim.
pass (green)	BLOCKED	The tests describe no new behaviour.
unverified	BLOCKED	The suite never ran. Make <code>commands.test</code> runnable.

test phase report

Tests: tests/test_ops.py. 4 tests written.
Status: fail (red). Committed: yes.

The committed tests are the contract, and execute may not rewrite it

TEST PROTECTION CONTRACT

- ✗ Never modify an existing assertion to make the code pass.
- ✗ Never delete a test.
- ✗ Never weaken a condition: exact match to substring, a dropped edge case, a relaxed mock count.
- + Add new tests for scenarios found while implementing.

Tie-breaker. If a "scaffolding fix" to a test file would change any assertion's expected value, it is not a scaffolding fix. Execute stops and escalates instead of editing.

WHAT ELSE IT DOES

- 3 Loads conventions and records a `### Conventions Applied` block citing each rule used.
- 7 Verification loop per Definition of Done item, then the full suite unless the policy is `on_end` or `never`.
- 8 Lint gate with `commands.lint`.
- 10 Test protection gate: compares test files in the working tree with the test phase's commit.

STOPS AND ASKS INSTEAD OF GUESSING WHEN

- a Definition of Done item is impossible as written,
- two items conflict,
- the work needs infrastructure outside the task's scope.

Six review dimensions, three severities, one verdict

DIMENSION	WHAT IT CHECKS
Spec compliance	Each Definition of Done item: satisfied, partial, missing. Undocumented deviations are findings.
Conventions	Each citation in Conventions Applied points at a real rule, and the code follows it.
Security and isolation	Authorization on new entry points, input validation, tenant scoping, no secrets.
Test integrity	Diffs every test file against the test phase's commit. Nothing weakened, deleted or re-expected.
Test quality	A test per Definition of Done item, error paths covered, and no production logic that exists only to satisfy a test.
Checklist	Spot-checks execute's Implementation Manifest, and scans <code>paths.review_checklist</code> if set.

Severity

Blocker must be fixed: security hole, cross-account leak, unmet Definition of Done item, weakened test.

Warning should be fixed: uncovered area, minor convention deviation.

Suggestion doesn't affect the verdict.

```
code-review report
```

Verdict: **CHANGES_REQUESTED**.

Blockers: DoD 2 unmet (float input accepted).

Warnings: none.

One blocker means no approval. **CHANGES_REQUESTED** stops the pipeline. It does not loop back to execute on its own; a person decides with `resume`.

end ships only what code-review approved

- 1 Detect changed areas from the diff.
- 2 Lint. Re-run `commands.test_all` if autofix could change behaviour, and always under `on_end`.
- 3 Persist discoveries: project facts go to the conventions file's discoveries span; phase quirks go to `.tdd-pipeline/memory/<phase>/`.
- 4 Commit and push the branch, staging named paths only.
- 5 Open the pull request against the base branch. Skipped if `pr.enabled: false`, there's no remote, or one already exists.
- 6 Update the tracker to its review state.
- 8 Cleanup hint: the `git worktree remove` command, to run only after the PR merges.

PULL REQUEST BODY

Summary of what changed.

Definition of Done copied from the spec, with satisfied items checked.

Test plan: the commands a reviewer runs, `commands.test_all`, `commands.lint`, and anything else the spec names.

end phase report

Lint: clean.
Persisted: 1 discovery to CONVENTIONS.md.
Pushed: TASK-1.
PR: <https://github.com/.../pull/57>
End phase: completed.

Every run ends with one of three results

SHIPPED

The end phase reached its last step. The branch is pushed and the PR is open.

WHAT YOU DO

Review and merge the PR like any other.
Remove the worktree afterwards.

STOPPED

- code-review returned `CHANGES_REQUESTED`
- execute stopped or escalated
- the worktree guard or setup script failed
- end stopped partway (push, tracker)

WHAT YOU DO

Read the reason and the handoff log, fix the cause, then `resume TASK-1`.

BLOCKED

- test reported `pass (green)`
- test reported `unverified`
- the spec has no `review` entry

WHAT YOU DO

Make `commands.test` runnable, run `review` on the spec, or rethink what the tests should describe.

A non-shipped report always carries a one-line reason, the branch and the worktree path, so you know where to pick up.

Phases talk to each other only through the spec's handoff log

ILLUSTRATIVE

```
TASK-1.md · Agent Handoff Log

### review (2026-09-10)
- Spec named calc.py; add() lives in ledger/ops.py.
  Fixed Files to Modify.

### test (2026-09-10)
- 4 tests in tests/test_ops.py, red on TypeError.
- test_add_ints_unchanged: regression guard,
  green by design.

### execute (2026-09-10)
- Validation in ledger/ops.py; no test file touched.
- Conventions Applied: raise, never return None
  (ledger/io.py:41)

### code-review (2026-09-10)
- Verdict: APPROVED. Blockers: none.
```

Each phase reads the whole log before it starts and appends one dated entry before it finishes. Entries are never edited. Useful notes are concrete: "changing this identifier breaks six tests", not "be careful".

WHERE A FINDING GOES

Fact about the codebase	conventions · discoveries span
Quirk of running a phase	.tdd-pipeline/memory/<phase>/
Detail of this task only	the handoff log
Your personal preference	your own memory, not the repo

When a finding could go in either the conventions file or the log, the conventions file wins.

resume runs exactly one phase and stops

```
/tdd:resume TASK-1          # infer the phase from the log
/tdd:resume TASK-1 execute   # or name it
```

HOW IT PICKS THE PHASE

- 1 Find the last `test`, `execute`, `code-review` or `end` entry, by position in the file.
- 2 None yet → `test`.
- 3 `code-review` with `CHANGES_REQUESTED` → `execute`.
- 4 `end` → nothing to resume. It never re-runs end blind.
- 5 Otherwise → the next phase in order.

A REJECTED REVIEW, FIXED

handoff log headings

```
### review
### test
### code-review  CHANGES_REQUESTED
    → resume picks execute
### execute
    → resume picks code-review
```

Each resume is one deliberate step by a person. After the fix, the next resume runs code-review, so the fix gets reviewed before anything ships.

The branch name is how resume finds a task. It looks up `refs/heads/TASK-1`. Rename the branch and resume will say there is nothing to resume.

Not every piece of work needs the full pipeline

plan

Resolves a task ID, settles open questions with you, drafts and refutes the spec, commits it. Runs nothing; task picks it up later.

```
/tdd:plan "validate input types on add"
```

review

Critiques a spec against the actual code and fixes it in place. Writes the `### review` entry the pipeline requires.

```
/tdd:review TASK-3
```

fix-bug

Failing test first, then root cause, minimal fix, verify. No spec; stops before pushing because nothing reviewed it.

```
/tdd:fix-bug "500 on empty list"
```

TRACKER.TYPE	TASK IDS COME FROM	STATUS UPDATES	NEEDS
none	Highest <code>TASK-N</code> in the specs directory + 1, confirmed with you	none	nothing
github	An existing issue, or one created with <code>gh issue create</code>	Labels <code>TASK:in-progress</code> → <code>TASK:in-review</code>	<code>gh</code> authenticated
linear	An existing issue, or a new one in <code>tracker.team</code>	The states in <code>tracker.states</code>	Linear MCP connected

Four constraints to know before you rely on it

The branch name is the task ID

No `feature/` prefix or username. Team branch naming conventions aren't supported yet, and a renamed branch is invisible to resume.

Three agents run on the larger model

`test`, `execute` and `code-review` are pinned to `opus` ; the orchestrator and end use `sonnet` . A full task runs all five. Change it in the frontmatter under `agents/` .

It commits on your current branch

`init`, conventions and the task spec are committed to the base branch, because worktrees only see committed files. Only named paths are staged, so your own uncommitted work stays uncommitted.

It stops rather than degrading

A missing key, an unauthenticated tracker or a suite it can't run each stop the run with the exact fix. Plan for a few stops while you set up a new project.

Your first task in five minutes

terminal

```
# 1. install (Claude Code)
claude plugin marketplace add dfrnks/agent-plugins
claude plugin install tdd@dfrnks

# 2. bootstrap: confirm the config, review each rule group
/tdd:init

# 3. every row pass or n/a
/tdd:doctor

# 4. optional: design first when the approach isn't settled
/tdd:plan "validate input types on add"
#   → TASK-1, spec at .tdd-pipeline/tasks/TASK-1.md
/tdd:review TASK-1

# 5. run it: task reuses the TASK-1 spec, reviews it inline,
#   stops at the gate (skipped step 4? pass a description)
/tdd:task TASK-1
#   → gate: proceed

# 6. only if something was interrupted or rejected
/tdd:resume TASK-1
```

WHERE THINGS LIVE IN YOUR PROJECT

.tdd-pipeline/config.yaml	the only required file
.tdd-pipeline/tasks/TASK-1.md	spec and handoff log, from plan or task
.tdd-pipeline/worktrees/TASK-1/	one per task, git-ignored
.tdd-pipeline/memory/<phase>/	phase-workflow learnings
paths.conventions	the rules that get enforced

On Cursor, the same commands use a dash: `/tdd-init`, `/tdd-plan`, `/tdd-review`, `/tdd-task`.

Source and docs: github.com/dfrnks/agent-plugins. The README covers permissions, updating and the full config schema.